Workshop 2026 — Groupe 6

PROJECT RESILIENCE-ALPHA · Dossier de gouvernance — état au 22/10/2026

Organisation de l'équipe

Répartition des domaines (WOR-18)

Chaque pilier a un responsable. Les décisions d'architecture restent prises ensemble.

Domaine	Responsable	Projet Linear	Tickets assignés
Développement	Michael	Développement — SI de paiement & remédiation	WOR-33 à WOR-42 (10)
Infrastructure	Reza	Infrastructure & Observabilité	WOR-43 à WOR-49 (7)
IA / Data	Léa	IA / Data — Détection ML & Assistant RAG	WOR-50 à WOR-57 (8)
Cybersécurité	Axel	Cybersécurité — Red Team & Défense	WOR-58 à WOR-66 (9)
Gouvernance / Transformation	Cédric	Gouvernance & Transformation	WOR-67 à WOR-73 (7)

Avancement du pilier Gouvernance au 22/10/2026

Ticket	Sujet	Statut	Livrable
WOR-31	Matrice de risques EBIOS RM, version initiale	Terminé, validé par les coachs	Partie « EBIOS RM »
WOR-68	Services critiques et coût d'un arrêt de paiement	Terminé	Partie « Services critiques »
WOR-69	Analyse NIS2 / DORA	Terminé, version 0.3	Partie « Analyse NIS2 / DORA »
WOR-70	Objectifs RTO / RPO	Terminé, version 0.3	Partie « Objectifs RTO / RPO » ; attaque A reprise en ≈ 40 s
WOR-71	PCA / PRA	Terminé, version 0.3	Partie « PCA / PRA » ; à confirmer par les essais (WOR-75, WOR-76, WOR-78)
WOR-72	Recommandations et arbitrages pour une direction	Terminé, version 1.0	Partie « Recommandations à la direction »
WOR-67	EBIOS RM complet (ateliers 3 à 5)	Terminé	Partie « EBIOS RM », ateliers 3 à 5
WOR-73	Relier EBIOS et PCA/PRA à l'architecture et aux essais	Terminé	Partie « État d'implémentation » (v0.5) ; résultats des répétitions intégrés

Décisions en attente avec les autres piliers

La gouvernance décrit le code tel qu'il est au 22/10. Les écarts à trancher avec l'équipe :

- **API du socle en mémoire** : un redémarrage ou une restauration efface les transactions. Le 0 transaction perdue des essais tient parce que la reprise s'est faite sans redémarrage. Brancher l'API sur PostgreSQL reste à faire (Reza, Michael).
- **Restauration automatique après une injection**, sans validation humaine ni second signal, déclenchée même quand le WAF a bloqué l'injection : à garder pour la démo, ou à encadrer (équipe).
- **Visages enrôlés** : consentement écrit et effacement après la démo (Axel, Cédric).

- **Attaque B** à diriger vers une route `/v1/…` existante, puis à chronométrer (Axel, Léa).

EBIOS RM — Analyse de risques

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans* `constat/etat-implementation.md`*.*

> *Ticket : WOR-31 · Responsable : Cédric · Version 0.3 du 21/10/2026*
>
> *Ateliers 1 et 2 : version 0.1,* ***validée par les coachs*** *(WOR-32). Ateliers 3 à 5 : ajoutés en version 0.2 (WOR-67), sur les composants réels relevés dans* `constat/etat-implementation.md`*.*

Hypothèses de travail

- L'entreprise est un **opérateur de paiement fictif**. Il traite les paiements de commerçants partenaires pour des utilisateurs finaux. Toutes les données sont fictives, sauf les empreintes faciales des membres de l'équipe enrôlés pour la double authentification du back-office.
- Le démonstrateur tourne sur **un seul PC Hôte**, avec Docker Compose sur le réseau `RESILIENCE-G6`. L'attaque vient d'un PC Attaquant distinct.
- Le coût d'un arrêt est chiffré dans `services-critiques.md` (WOR-68). Les objectifs RTO/RPO sont fixés dans `rto-rpo.md` (WOR-70).

Atelier 1 — Cadrage et socle de sécurité

1.1 Valeurs métier

ID	Valeur métier	Nature	Responsable métier (fictif)
VM1	Traitement des paiements : initier, autoriser, enregistrer une transaction	Processus	Direction des opérations
VM2	Consultation des transactions par les partenaires et les utilisateurs	Processus	Direction des opérations
VM3	Authentification et gestion des accès	Processus	RSSI
VM4	Données de comptes et de transactions	Information	DPO / Direction financière
VM5	Traçabilité : journaux et preuves d'audit	Information	RSSI / Conformité
VM6	Capacité de détection et de reprise automatique	Processus	Direction de la production

1.2 Biens supports

ID	Bien support	Valeurs métier portées
BS1	Front web de paiement	VM1, VM2
BS2	API métier de paiement (Node.js, `services/payment-api/backend/`)	VM1, VM2, VM4
BS3	Authentification intégrée à l'API (pas de service séparé), sessions en base ; dans le socle, elles sont en mémoire tant que l'API n'est pas branchée sur PostgreSQL	VM3
BS4	Base PostgreSQL (utilisateurs, sessions, intentions de paiement)	VM1, VM4
BS5	WAF / reverse proxy	VM1, VM2, VM3
BS6	Stack d'observabilité (Prometheus, Loki, Grafana) et SIEM	VM5, VM6
BS7	Service d'inférence ML (détection d'anomalies)	VM6
BS8	Moteur de remédiation, playbooks Ansible, accès au moteur Docker	VM6, puis indirectement toutes
BS9	Gestion des secrets (HashiCorp Vault)	VM3, VM4
BS10	Assistant RAG (Ollama, LangChain)	VM6 (aide aux opérateurs)

ID	Bien support	Valeurs métier portées
BS11	PC Hôte et réseau `RESILIENCE-G6`	Toutes
BS12	Dépôt Git et procédure de déploiement	VM6

> ***Point d'attention :*** *BS8 (le moteur de remédiation) a des droits élevés sur tous les conteneurs. C'est le bien support le plus sensible du dispositif (voir ER6).*
>
> ***Alignement sur le code (20/10) :*** *BS7 (inférence ML) et BS8 (remédiation) sont codés ; BS8 tourne sur l'hôte avec les droits Docker. BS5 (WAF) est déployé et l'API est derrière lui. Dans le socle, l'API garde ses données en mémoire (BS4 n'y est pas utilisée). Les vulnérabilités relevées dans le code (port de la base publié, secrets en clair, code de vérification renvoyé par l'API, appel à OpenAI) sont listées dans `constat/etat-implementation.md` §3 et sont traitées dans les ateliers 4 et 5.*

1.3 Événements redoutés

Échelle de gravité : **G1** mineure · **G2** significative · **G3** grave · **G4** critique.

Pour une indisponibilité, l'échelle est chiffrée dans `services-critiques.md` (WOR-68) : G1 ≤ 5 min (~1,5 k€), G2 ≤ 30 min (~10 k€), G3 ≤ 2 h (~700 k€, SLA dépassé), G4 > 2 h (> 1,5 M€, incident majeur DORA).

ID	Événement redouté	VM	Impacts	Gravité
ER1	Paiements indisponibles au-delà du RTO	VM1	Transactions des partenaires bloquées, perte de chiffre d'affaires, perte de confiance, incident majeur à déclarer (DORA)	**G4**
ER2	Transactions altérées ou incohérentes (montant modifié, double débit, transaction fantôme)	VM1, VM4	Préjudice financier direct, litiges, données comptables fausses	**G4**
ER3	Fuite des données de comptes ou de transactions	VM4	Atteinte à la vie privée (RGPD), sanctions, atteinte à l'image	**G3**
ER4	Accès non autorisé à un compte (usurpation d'identité)	VM3	Fraude, opérations frauduleuses initiées	**G3**
ER5	Journaux perdus ou altérés	VM5	Investigation impossible, preuves perdues, non-conformité	**G3**
ER6	Moteur de remédiation détourné ou en erreur (arrêt de services, restauration de mauvaises données)	VM6 → VM1	Interruption provoquée par le dispositif de défense lui-même, possibilité de rebond sur tout le SI	**G4**
ER7	Consultation des transactions indisponible	VM2	Gêne pour les partenaires, appels au support	**G2**

1.4 Socle de sécurité

Référentiel	Applicabilité	Écarts connus à ce stade
DORA (règlement UE 2022/2554)	Applicable : entité financière	Pas de tests de résilience menés par un tiers, pas de gestion des prestataires TIC (voir WOR-69)
NIS2 (directive UE 2022/2555)	Non applicable : les établissements de paiement ne sont pas à l'annexe I. Pour les entités financières, DORA prime de toute façon	Voir `nis2-dora.md` (WOR-69)
RGPD	Applicable par principe (données personnelles fictives)	Aucune donnée réelle, donc écart sans conséquence pour le prototype
Guide d'hygiène informatique de l'ANSSI	Bonnes pratiques de référence	Hôte unique (point de défaillance unique), pas de sauvegarde hors site
PCI DSS	Référence du secteur pour les données de carte	**Hors périmètre** : aucune donnée de carte manipulée

Atelier 2 — Sources de risque et objectifs visés

Cotation de la pertinence : **1** faible · **2** moyenne · **3** forte. Elle combine la motivation, les ressources et l'activité constatée de la source.

ID	Source de risque (SR)	Objectif visé (OV)	Motivation	Ressources	Pertinence	Retenu ?
SR1	Cybercriminel organisé	Gain financier : fraude aux paiements, rançongiciel	Forte	Importantes	**3**	✅
SR2	Hacktiviste	Visibilité, nuire à l'image : déni de service	Moyenne	Moyennes (botnets loués)	**3**	✅
SR3	Initié malveillant (employé, prestataire)	Vengeance ou gain : sabotage, fraude interne	Moyenne	Accès légitimes	**2**	✅
SR4	Attaquant opportuniste	Défi, revente d'accès : exploitation d'une vulnérabilité web, bourrage d'identifiants	Faible	Outils publics	**2**	✅
SR5	Concurrent ou officine d'espionnage	Vol de données commerciales	Faible	Moyennes	1	❌ données fictives, sans valeur concurrentielle
SR6	Acteur étatique (APT)	Déstabiliser le secteur financier	Faible pour une petite entité	Très importantes	1	❌ hors de portée d'un prototype de 4 jours, cité comme risque résiduel

> *Les événements non intentionnels (erreur d'exploitation, panne, faux positif du modèle ML) ne sont pas des sources de risque au sens d'EBIOS. Ils seront traités dans le PCA/PRA (WOR-71).*

Matrice de risques initiale

Vraisemblance provisoire : **V1** peu vraisemblable · **V2** vraisemblable · **V3** très vraisemblable · **V4** quasi certain. Elle est affinée dans l'atelier 4 ; le risque résiduel est dans l'atelier 5.

ID	Risque (SR → mode d'action → ER)	Gravité	Vraisemblance	Niveau
R1	SR2 → déni de service applicatif sur l'API ou le front → ER1	G4	V3	🔴 Élevé
R2	SR1 → injection applicative sur l'API → ER2	G4	V2	🔴 Élevé
R3	SR3 → détournement des droits du moteur de remédiation → ER6	G4	V2	🔴 Élevé
R4	SR4 → bourrage d'identifiants ou force brute sur l'authentification → ER4	G3	V3	🟠 Significatif
R5	SR1 → compromission puis exfiltration de la base → ER3	G3	V2	🟠 Significatif
R6	SR3 → effacement ou altération des journaux → ER5	G3	V2	🟠 Significatif
R7	SR1 → rançongiciel sur le PC Hôte → ER1, ER3	G4	V1	🟠 Significatif
R8	SR2 → saturation de la consultation → ER7	G2	V3	🟡 Modéré

Cartographie

Gravité \ Vraisemblance	V1	V2	V3	V4
G4	R7	R2, R3	**R1**	
G3		R5, R6	R4	
G2			R8	

Gravité \ Vraisemblance	V1	V2	V3	V4
G1				

Lecture

- **R1 (déni de service)** est le risque le plus élevé. C'est aussi le meilleur candidat pour le scénario d'attaque de la démo (WOR-29) : effet observable, détectable par le ML et réparable automatiquement.
- **R2 et R5** (injection, exfiltration) appellent une réponse différente d'une simple restauration de la disponibilité. Le sujet demande explicitement de ne pas confondre les deux (voir WOR-39).
- **R3** porte sur notre propre dispositif de défense. Il justifie WOR-64 (sécuriser le moteur de remédiation), que le jury évalue sur l'Axe 2.

Atelier 3 — Scénarios stratégiques

3.1 Écosystème

Les parties prenantes par lesquelles une attaque peut passer. Chaque critère est coté de 1 (faible) à 4 (fort). **Menace = (dépendance × pénétration) / (maturité cyber × confiance)** ; au-delà de 2, la partie prenante est jugée critique. Les cotes sont des estimations pour l'entreprise fictive.

ID	Partie prenante	Accès au système	Dép.	Pén.	Mat.	Conf.	Menace
PP1	Partenaires commerçants (200) et utilisateurs finaux	API derrière le WAF, comptes clients	4	2	1	1	**8** critique
PP2	Autres postes du réseau `RESILIENCE-G6`	Ports exposés de l'hôte : WAF (8080), base de l'application (55432)	1	3	1	1	**3** critique
PP3	Exploitants (équipe projet)	Comptes de l'hôte, droits Docker, Vault, dépôt Git	4	4	2	3	**2,7** critique
PP5	Service d'inférence ML	Clé HMAC qui autorise les ordres au moteur de remédiation	4	3	2	3	**2** critique
PP4	Prestataires TIC : OpenAI, registres d'images, dépendances npm et Python	Appels sortants, code tiers exécuté dans les conteneurs	2	2	3	2	0,7

Mesures sur l'écosystème : filtrage et blocage des clients au WAF (PP1, PP2), signature des alertes (PP5), registre des prestataires TIC (PP4, REC-11). Le cloisonnement des exploitants (PP3) n'est pas traité dans le prototype.

3.2 Scénarios stratégiques

ID	Source → objectif	Chemin dans l'écosystème	Événement redouté	Gravité
SS1	SR2 hacktiviste → nuire à l'image	Depuis le réseau (PP2) ou en se faisant passer pour des clients (PP1), saturer l'API	ER1 paiements indisponibles	G4
SS2	SR1 cybercriminel → gain financier	Compte client (PP1) → injection dans l'API → transactions modifiées	ER2 transactions altérées	G4
SS3	SR1 cybercriminel → revente de données	Réseau (PP2) → base de l'application exposée	ER3 fuite de données	G3
SS4	SR3 initié → sabotage	Droits d'exploitant (PP3) ou service d'inférence compromis (PP5) → ordres au moteur de remédiation	ER6 défense détournée, puis ER1 et ER2	G4
SS5	SR4 opportuniste → fraude	Comptes clients (PP1) → bourrage d'identifiants → paiements	ER4 usurpation	G3
SS6	SR3 initié → effacer ses traces	Droits d'exploitant (PP3) → remise à l'état sain ou purge de Loki	ER5 journaux perdus	G3

Atelier 4 — Scénarios opérationnels

Chaque scénario suit les composants réels du démonstrateur (`constat/etat-implementation.md`). La vraisemblance tient compte des mesures **déjà codées**.

Scénario	Étapes de l'attaque	Ce qui la freine aujourd'hui	Vraisemblance
SO1 (SS1, R1) Déni de service	1. Rafale de requêtes depuis le PC Attaquant vers le WAF (port 8080) · 2. L'API, limitée à 0,75 CPU, sature ou renvoie des erreurs · 3. Les paiements légitimes échouent	Limite de 60 requêtes par minute et par client ; détection ML, puis blocage en 403 des IP qui pèsent au moins 20 % parmi les 5 plus actives ; redémarrage si besoin	**V3** : outil public. Une attaque répartie sur beaucoup d'IP perd 5 sources par alerte, mais les autres continuent
SO2 (SS2, R2) Injection	1. Requêtes avec motifs d'injection vers une route `/v1/…` · 2. Une variante passe le WAF · 3. Elle atteint le stockage de l'API	OWASP CRS bloquant : 9 vecteurs sur 10 en 403 ; détection de la famille `injection` ; blocage de la source	**V1** : le WAF arrête presque tout, et l'API du socle n'a pas de base. Réserve : l'attaque B vise encore `/api/pay`, qui n'existe pas ; le chemin jusqu'à l'API n'a pas été essayé
SO2b (SS2, R2) Restauration provoquée	1. Une seule requête d'injection, même bloquée par le WAF · 2. La détection lève une alerte `sqli` légitime · 3. Le moteur restaure : toutes les transactions sont effacées	Aucune : le moteur restaure sans vérifier que l'injection a réussi	**V3** : une requête suffit (`ml/training/README.md`). Le 21/10, deux fausses alertes ont produit le même effet sans attaque
SO3a (SS4, R3) Restauration abusive	1. Accès à l'hôte, ou compromission du service d'inférence, qui détient la clé HMAC · 2. Alerte `sqli` forgée et signée · 3. Le moteur restaure sans validation humaine : toutes les transactions sont effacées	Signature HMAC, refus des alertes anciennes ou rejouées, liste fermée de playbooks, écoute sur la loopback et la passerelle Docker (jamais sur le réseau)	**V2** : la clé et les droits Docker sont sur l'hôte, sans cloisonnement entre exploitants
SO3b (SS4, R3) Blocage de clients légitimes	Même accès · alerte `dos` forgée avec les IP des partenaires	Les IP de l'hôte ne sont jamais bloquées ; rien ne protège celles des partenaires	**V2**
SO4 (SS3, R5) Exfiltration	1. Depuis le réseau, connexion au port 55432 · 2. Mot de passe lu dans le dépôt · 3. Copie de la base	Aucune, si le déploiement autonome de Pay&Go (`services/payment-api/infra/`) tourne	**V3** : trivial. Sans objet pour le socle, qui n'expose que le WAF au réseau
SO5 (SS5, R4) Usurpation	1. Essais d'identifiants sur `/v1/auth/login` · 2. Session obtenue · 3. Paiement validé avec le code renvoyé par l'API (`demoCode`)	12 essais par 15 min et par client ; **identifiant verrouillé après 2 échecs** (5 min, puis 10, 15…) ; visage exigé avant le mot de passe pour les comptes du back-office enrôlés	**V1** : deviner un mot de passe devient impraticable. Restent la réutilisation d'un mot de passe connu (un essai par compte) et un code de paiement qui ne protège rien
SO5b (SS5, R4) Usurpation d'un administrateur sur place	1. Accès physique au poste du back-office (la consigne additionnelle 1 cite des attaques physiques) · 2. Photo de l'administrateur devant la caméra · 3. Mot de passe obtenu par ailleurs	Visage **et** mot de passe exigés ; 2 échecs verrouillent l'identifiant ; LED rouge visible sur place	**V2** : le modèle ne vérifie pas qu'il s'agit d'un visage vivant, une photo peut suffire. Il faut quand même le mot de passe
SO6 (SS6, R6) Effacement des journaux	1. Exploitant malveillant sur l'hôte · 2. Remise à l'état sain ou purge des volumes · 3. Les journaux de Loki disparaissent	Export des journaux avec empreintes (`scripts/export-logs.sh`), mais à lancer à la main	**V2**

R7 (rançongiciel, V1) et R8 (même chemin que SO1) ne sont pas détaillés.

Atelier 5 — Traitement du risque

Règle des niveaux, identique à la matrice initiale : 🔴 si G4 et V2 ou plus ; 🟠 si G4·V1, ou G3 et V2 ou plus ; 🟡 sinon.

5.1 Mesures et risque résiduel

Démo = mesure à faire avant les essais ; sinon, mesure pour un contexte industriel (`recommandations.md`).

Risque	Mesures en place	Résiduel actuel	Mesures complémentaires	Résiduel visé
R1 déni de service	Limite par client, blocage au WAF, redémarrage vérifié par une transaction	G2·V2 🟡, sous réserve du RTO mesuré (WOR-79) ; un partenaire bloqué à tort le reste jusqu'à une action manuelle	**Démo :** levée automatique des blocages. Industriel : protection en amont (REC-10)	G2·V2 🟡
R2 injection	WAF CRS bloquant, détection, blocage, retour à l'état sain	G4·V3 🔴 : une seule tentative d'injection, même bloquée, déclenche la restauration qui efface toutes les transactions (SO2b)	**Démo :** ne pas restaurer quand le WAF a tout bloqué, API sur PostgreSQL, sauvegarde avant essai, attaque B recâblée. Industriel : correction du code, restauration à un instant précis (REC-6)	G3·V1 🟡 avec REC-6
R3 remédiation détournée	Signature HMAC, liste fermée, garde-fous, écoute hors réseau	G4·V2 🔴	**Démo :** moteur en conteneur, sans droits Docker complets. Industriel : REC-2, REC-3, REC-4	G4·V1 🟠
R4 usurpation	Limite de 12 essais par 15 min, verrouillage progressif de l'identifiant, visage exigé pour le back-office	G3·V2 🟠 : un visage en photo peut passer (SO5b) ; deviner un mot de passe est devenu impraticable (SO5)	**Démo :** ne plus renvoyer le code de vérification. Industriel : double authentification pour les clients, détection du vivant pour la reconnaissance faciale (SO5b)	G3·V1 🟡
R5 exfiltration	Socle : seul le WAF est exposé au réseau	G3·V3 🟠 si le déploiement autonome de Pay&Go tourne	**Démo :** fermer le port 55432, secrets de l'application dans Vault	G3·V1 🟡
R6 journaux	Export avec empreintes, Loki 7 jours	G3·V2 🟠	**Démo :** export appelé avant toute restauration ou remise à l'état sain. Industriel : journaux hors de l'hôte, immuables	G3·V1 🟡
R7 rançongiciel	Aucune	G4·V1 🟠	Industriel : sauvegardes hors site, deuxième site (REC-5, REC-9)	G4·V1 🟠
R8 consultation	Même chaîne que R1	G2·V2 🟡	—	G2·V2 🟡

5.2 Cartographie du risque résiduel

Gravité \ Vraisemblance	V1	V2	V3	V4
G4	R7	**R3**	**R2**	
G3		R4, R6	R5	
G2		R1, R8		

Actuel. Une fois les mesures « démo » faites : R2 passe en G4·V1, R3 en G4·V1, R5 et R6 en G3·V1 ; R4 attend la détection du vivant pour passer en G3·V1.

5.3 Acceptation du risque résiduel

Risque	Décision	Par qui	Jusqu'à quand
R3	Accepté pour la démo : la restauration automatique sans validation humaine est un choix de démonstration	Direction générale (fictive), à la présentation finale	Mise en production, qui exige REC-2, REC-3 et REC-4
R2	Accepté, une fois la restauration réservée aux injections qui passent le WAF : perte des transactions à chaque restauration	Idem	Mise en place de REC-6
R7	Accepté : hôte unique, sans sauvegarde hors site	Idem	Mise en place de REC-5 et REC-9
R1, R8	Accepté en G2, si le RTO mesuré le confirme	Idem	Revue après les essais (WOR-79)

5.4 Plan d'amélioration

Mesure	Risques	Responsable	Échéance	Comment on le vérifie
API du socle sur PostgreSQL, `pg_dump` avant chaque essai	R2, RPO	Reza, Michael	Avant les essais	Comptage des transactions avant et après restauration (WOR-75)
Attaque B sur une route `/v1/…`	R2	Axel	Avant les essais	Répétitions (WOR-78)
Pas de restauration quand le WAF a bloqué toutes les requêtes d'injection	R2	Michael, Léa	Avant les essais	Une requête d'injection bloquée : blocage de la source, aucune restauration
Consentement écrit des personnes enrôlées, effacement des visages après la démo	ER3, RGPD	Cédric, Axel	Avant la démo / après la démo	Formulaires signés ; volume `face-data` purgé
Modèle évalué sur des fenêtres d'attaque	R1, R2, R3	Léa	Avant les essais	Rapport d'entraînement : taux et délai de détection (WOR-52)
Code de vérification non renvoyé	R4	Michael	Avant les essais	Appel à l'API
Port 55432 fermé, secrets de l'application dans Vault	R5	Axel	Avant les essais	Scan de ports depuis le PC Attaquant
Export des journaux avant restauration et remise à l'état sain	R6	Michael, Reza	Avant les essais	Dossier de preuves après chaque essai
Levée automatique des blocages	R1	Michael	Fin du workshop	Chaos engineering (WOR-76)
Moteur en conteneur, droits limités	R3	Michael, Axel	Fin du workshop	Revue des droits (WOR-64)
REC-2 à REC-6, REC-9 à REC-11	Tous	Direction (fictive)	6 à 12 mois	Voir `recommandations.md`

Services critiques et coût d'un arrêt de paiement

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans `constat/etat-implementation.md`.*

> *Ticket : WOR-68 · Responsable : Cédric · Version 0.2 du 22/10/2026*
>
> *Il fournit l'échelle de gravité de l'EBIOS RM, et sert d'entrée aux objectifs de reprise (`rto-rpo.md`) et au PCA/PRA.*

Hypothèses sur l'entreprise fictive

Chiffres **fictifs**, choisis pour un opérateur de paiement de taille moyenne.

Paramètre	Valeur retenue	Commentaire
Transactions par jour	1 200 000	Soit environ 830 par minute en moyenne
Montant moyen d'une transaction	45 €	Paiement de détail
Flux traité par jour	54 M€	1 200 000 × 45 €
Commission moyenne	0,8 %	Revenu de l'opérateur sur chaque transaction
Revenu par jour	432 k€	Soit environ 158 M€ par an
Facteur des heures de pointe	× 2,5	12 h–14 h et 18 h–20 h
Taux d'abandon définitif pendant un arrêt	40 %	Les autres transactions sont rejouées après la reprise
Commerçants partenaires	200	Liés par un contrat de niveau de service
Engagement de disponibilité (SLA)	99,9 % par mois	Soit 43 minutes d'arrêt tolérées par mois
Pénalité en cas de dépassement du SLA	5 % de la facture mensuelle	Environ 650 k€ si tous les partenaires sont touchés

Coût d'un arrêt du traitement des paiements

Perte directe par minute

	Moyenne	Heure de pointe
Transactions bloquées par minute	830	2 080
Flux bloqué par minute	37 500 €	93 750 €
Revenu bloqué par minute	300 €	750 €
Revenu perdu définitivement par minute (40 % d'abandon)	**120 €**	**300 €**

La perte directe reste modeste. **Le vrai coût vient des effets de seuil** : la pénalité contractuelle, l'obligation réglementaire et la perte de confiance.

Coût selon la durée (incident en heure de pointe)

Durée de l'arrêt	Perte directe	Effets de seuil	Coût total estimé	Gravité EBIOS
≤ 5 min	1 500 €	Aucun	~1,5 k€	G1 mineure
5 à 30 min	jusqu'à 9 k€	Hausse des appels au support	~10 k€	G2 significative
30 min à 2 h	jusqu'à 36 k€	Budget SLA mensuel de 43 min dépassé, donc pénalité d'environ 650 k€ ; incident probablement majeur au sens de DORA (critère économique)	~700 k€	G3 grave

Durée de l'arrêt	Perte directe	Effets de seuil	Coût total estimé	Gravité EBIOS
> 2 h	36 k€ et plus	Pénalité SLA, incident **majeur au sens de DORA** (notification à l'ACPR), départ estimé de 0,5 % des partenaires (environ 790 k€ par an)	> 1,5 M€	G4 critique

> *Le seuil DORA de 2 h vient du règlement délégué (UE) 2024/1772. Mais dès que la pénalité SLA tombe, l'impact économique dépasse 100 k€ : l'incident devient probablement majeur* ***dès 30 min*** *(`nis2-dora.md` §3).*

À retenir : l'arrêt coûte peu pendant quelques minutes, puis le coût explose. Une reprise manuelle (30 min et plus) fait basculer l'incident en G3 ; une reprise automatique en quelques minutes le maintient en G1.

Services critiques par ordre de priorité

Rang	Service	Criticité	Pourquoi	Dégradation acceptable
1	Traitement des transactions (API de paiement et base de données)	**Vitale**	Porte directement le revenu et l'engagement envers les partenaires	Aucune
2	Authentification	**Vitale**	Sans elle, aucune transaction ne peut être initiée. Intégrée à l'API : elle tombe avec elle. Le back-office dépend en plus de la reconnaissance faciale (consigne additionnelle 1)	Aucune pour les clients ; pour le back-office, quelques minutes, avec une connexion de secours à prévoir
3	WAF / point d'entrée	**Vitale**	Point de passage unique de tout le trafic	Un contournement temporaire n'est **pas** acceptable (exposition directe)
4	Journalisation et SIEM	Importante	Aucun revenu en jeu, mais elle est indispensable à l'investigation et à la conformité	Arrêt de quelques heures, **sans perte** de journaux
5	Consultation de l'historique	Importante	Gêne pour les partenaires, sans perte de revenu	Mode dégradé possible (lecture différée)
6	Supervision, détection ML, moteur de remédiation	Support critique	N'est pas un service métier, mais conditionne la reprise de tous les autres	Arrêt de courte durée, avec retour à une surveillance manuelle
7	Assistant RAG	Non critique	Aide les opérateurs ; le service rendu aux clients n'en dépend pas	Indisponibilité acceptable

Ce qu'on protège en priorité

1. **La continuité du traitement des paiements** (rangs 1 à 3) : la reprise automatique doit viser un retour en service en moins de 5 minutes.
2. **L'intégrité des transactions déjà validées** : une transaction confirmée ne doit jamais être perdue ni dupliquée. Une transaction perdue coûte bien plus cher qu'une minute d'arrêt (litiges, écritures comptables fausses).
3. **La conservation des journaux** : sans eux, impossible d'expliquer l'incident au régulateur ni d'améliorer la détection.

Objectifs de reprise par service et protocole de mesure : `rto-rpo.md`.

Analyse NIS2 / DORA

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans `constat/etat-implementation.md`.*

> *Ticket : WOR-69 · Responsable : Cédric · Version 0.3 du 22/10/2026*
>
> *Le sujet demande de **distinguer les exigences pertinentes des obligations qu'un prototype de quatre jours ne peut pas démontrer**. Pour chaque exigence, ce document dit donc ce que le démonstrateur prouve, et ce qu'il ne prouve pas.*

1. Quel texte s'applique ?

Question	Réponse	Fondement
Notre entreprise fictive est-elle dans le champ de DORA ?	**Oui.** Un opérateur de paiement est un établissement de paiement, explicitement visé.	DORA (règlement UE 2022/2554), art. 2.1 b. Applicable depuis le 17/01/2025.
Est-elle dans le champ de NIS2 ?	**Non.** Les établissements de paiement ne figurent pas à l'annexe I de NIS2, dont le secteur bancaire ne vise que les établissements de crédit.	NIS2 (directive UE 2022/2555), annexe I.
Si les deux s'appliquaient, lequel prime ?	**DORA.** C'est un acte sectoriel qui prévoit des mesures au moins équivalentes ; il remplace NIS2 pour la gestion du risque et la notification des incidents.	NIS2, art. 4 et considérant 28.
Qui est l'autorité compétente ?	L'**ACPR** pour un établissement de paiement en France.	Désignation nationale au titre de DORA.
Le cadre simplifié (art. 16) s'applique-t-il ?	**Non.** Avec 158 M€ de revenu annuel et 200 partenaires, l'entreprise n'est pas une petite structure exemptée.	DORA, art. 16.

Conclusion : on raisonne en DORA. NIS2 reste utile comme grille de lecture (mesures de l'art. 21, chaîne d'approvisionnement), mais ce n'est pas le texte qui nous oblige.

2. Ce que DORA exige, et ce que le démonstrateur prouve

✅ démontré · 🟡 démontré en partie · ❌ non démontrable en 4 jours. Les preuves seront confirmées par les essais (WOR-73).

2.1 Gestion du risque TIC (art. 5 à 16)

Exigence	Art.	Ce que le prototype montre (preuve)	Statut
Cadre de gestion du risque documenté, fonctions critiques identifiées	6, 8	EBIOS RM, services critiques chiffrés, biens supports reliés aux valeurs métier	✅
Protection : moindre privilège, secrets, filtrage	9	WAF bloquant, secrets du socle dans Vault, conteneurs durcis, moteur de remédiation encadré ; authentification forte du back-office (visage puis mot de passe, verrouillage après 2 échecs), comme le demande l'art. 9.4 d)	🟡 un seul hôte ; secrets de Pay&Go et port de sa base encore exposés
Détection rapide des anomalies	10	Détection ML sur métriques et journaux ; alerte ≈ 30 s après le début de l'attaque A	✅
Réponse et rétablissement	11	**Cœur de la démo** : remédiation automatique, puis transaction de test réussie ; reprise en ≈ 40 s sur l'attaque A, sous le RTO de 5 min	✅ pour la disponibilité ; 🟡 pour l'intégrité (restauration qui perd les transactions)
Tests du plan de continuité	11.6	Scénarios rejoués et chronométrés	🟡 deux scénarios, pas de programme annuel

Exigence	Art.	Ce que le prototype montre (preuve)	Statut
Sauvegarde et restauration sans perte	12	Comptage des transactions avant et après reprise	❌ aucune sauvegarde ; la restauration repart des données de départ
Redondance	12.4	—	❌ hôte unique, assumé
Retour d'expérience	13	Analyse après incident à partir des journaux, aidée par l'assistant RAG	🟡
Communication de crise	14	Plan de communication du PCA	🟡 sur papier
Responsabilité de la direction, formation	5	—	❌ pas de direction réelle

2.2 Gestion et notification des incidents (art. 17 à 23)

Exigence	Article	Ce que le prototype montre	Statut
Processus de détection, journalisation et suivi des incidents	17	Chaque incident est horodaté dans le journal du moteur de remédiation et visible au tableau de bord ; blocages du WAF dans le SIEM	🟡
Classification selon les critères réglementaires	18	Grille appliquée au scénario de la démo (voir section 3)	✅ sur papier, à partir des mesures réelles
Notification des incidents majeurs à l'ACPR : initiale, intermédiaire, finale	19	Les journaux du SIEM contiennent les informations nécessaires. Aucune notification réelle n'est envoyée.	🟡
Information des clients touchés	19.3	—	❌

2.3 Tests de résilience (art. 24 à 27)

Exigence	Article	Ce que le prototype montre	Statut
Programme de tests : analyses de vulnérabilités, tests de scénarios, tests de performance	24, 25	Scénario Red Team depuis le PC Attaquant, scans de vulnérabilités	🟡 un essai ponctuel, pas un programme
Tests d'intrusion fondés sur la menace (TLPT), tous les 3 ans, par des testeurs indépendants	26, 27	—	❌ réservé aux entités désignées, exige des testeurs externes

2.4 Risque lié aux prestataires TIC (art. 28 à 44)

Exigence	Article	Ce que le prototype montre	Statut
Registre des accords avec les prestataires TIC	28.3	Inventaire des briques utilisées (images Docker, Ollama, modèles)	🟡 inventaire technique, pas contractuel
Clauses contractuelles obligatoires, stratégies de sortie	28.8, 30	—	❌ aucun contrat
Concentration du risque sur un prestataire	29	Tout est auto-hébergé, sauf l'assistant de discussion de l'application, qui appelle OpenAI	🟡 un prestataire externe à inscrire au registre, ou à remplacer par le modèle local (Ollama)

3. Le scénario de la démo est-il un incident majeur ?

DORA classe un incident comme **majeur** s'il touche un service critique et, en plus, soit entraîne une perte de données après un accès malveillant, soit dépasse les seuils d'au moins deux autres critères (règlement délégué (UE) 2024/1772). Les critères utiles ici :

- **Durée** : interruption d'un service critique de plus de 2 h (ou incident de plus de 24 h) ;
- **Clients touchés** : plus de 10 % des clients ou des contreparties financières ;

- **Impact économique** : plus de 100 k€ de coûts et pertes ;
- **Perte de données** : atteinte à la disponibilité, l'intégrité ou la confidentialité des données.

Appliqué au risque R1 (déni de service sur l'API, scénario principal), en heure de pointe et avec les 200 partenaires touchés :

Mode de reprise	Durée	Critères dépassés	Majeur ?	Conséquence
Automatique (cible de la démo)	≤ 5 min	Clients touchés	**Non**	Incident enregistré et classé, pas de notification
Manuelle	30 min à 2 h	Clients touchés, impact économique (pénalité SLA ~650 k€)	**Oui, probablement**	Notification initiale à l'ACPR, puis rapports intermédiaire et final
Manuelle, longue	> 2 h	Clients, durée, impact économique	**Oui**	Idem, et incident G4 dans EBIOS

> ***Attention :*** *le critère économique peut faire basculer l'incident en majeur* ***avant*** *le seuil de 2 h, dès que la pénalité SLA est déclenchée (43 min d'arrêt cumulé dans le mois).*
>
> ***Incidents récurrents :*** *des incidents qui ne sont pas majeurs un par un, mais qui ont la même cause apparente, se sont produits au moins deux fois en 6 mois et dépassent ensemble les seuils, sont traités comme un seul incident majeur (règlement délégué (UE) 2024/1772, art. 16). Des dénis de service courts et répétés peuvent donc finir par imposer une notification.*

Pour R2 et R5 (injection, exfiltration), le critère « perte de données après accès malveillant » suffit, avec la criticité du service, à rendre l'incident majeur, **quelle que soit la vitesse de reprise**. Restaurer la disponibilité ne dispense pas de notifier. C'est la distinction que le sujet demande de ne pas manquer.

Paradoxe du démonstrateur : la restauration actuelle efface toutes les transactions, et elle se déclenche même quand le WAF a bloqué l'injection (`constat/etat-implementation.md` §3). C'est alors notre propre remédiation qui crée la perte de données après une tentative malveillante, donc l'incident majeur. D'où la recommandation de ne restaurer que si une injection a passé le WAF.

Argument pour la direction : la reprise automatique en moins de 5 minutes n'est pas seulement un gain technique. Elle maintient un déni de service sous les seuils de DORA, ce qui évite la notification réglementaire et la pénalité contractuelle.

4. Ce que nous ne pouvons pas démontrer

Les limites du prototype face à DORA (hôte unique, sauvegardes, tests indépendants, contrats, direction, notification réelle) et ce qu'il faudrait en contexte industriel sont regroupées dans `recommandations.md` §3.

Objectifs RTO / RPO

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans `constat/etat-implementation.md`.*

> *Ticket : WOR-70 · Responsable : Cédric · Version 0.3 du 21/10/2026*
>
> *Objectifs fixés à partir du coût d'un arrêt (`services-critiques.md`) et des seuils de DORA (`nis2-dora.md`), avec **la façon de les mesurer** pendant les essais (WOR-79).*

1. Définitions retenues

- **RTO** (durée maximale d'interruption) : temps entre le **début de l'impact** (première transaction en échec) et la **première transaction réussie et vérifiée** après la reprise. On compte depuis l'impact, pas depuis la détection : c'est ce que subissent les partenaires, et ce que DORA mesure.
- **RPO** (perte de données maximale admissible) : quantité de **transactions confirmées au client** qui manquent ou sont dupliquées après la reprise. On l'exprime en nombre de transactions.
- **Reprise vérifiée** : API accessible, transaction test réussie, données cohérentes (WOR-77). Un conteneur redémarré ne compte pas.

2. Deux niveaux d'objectifs

Niveau	À quoi il sert	D'où il vient
Objectif métier	Ce que l'entreprise fictive exige ; c'est ce qu'on présente à la direction	Coût d'un arrêt : au-delà de 5 min, l'incident sort de G1 ; au-delà de 30 min, l'incident passe en G3 : le budget SLA de 43 min par mois est presque consommé (pénalité d'environ 650 k€ au-delà) et la notification DORA devient probable
Cible du démonstrateur	Ce qu'on vise et mesure sur le PC Hôte	Objectif métier, avec de la marge, et la contrainte du Clip Chrono (vidéo continue de 60 s exigée par le sujet) : 60 s en continu entre l'attaque et le service rétabli (WOR-83)

3. Objectifs par service

Service	Criticité	RTO métier	RPO métier	Cible démonstrateur	Mécanisme prévu
Traitement des transactions (API + BDD)	Vitale	**≤ 5 min**	**0** transaction confirmée perdue ou dupliquée	RTO ≤ 60 s (attaque A) · ≤ 3 min (attaque B)	Remédiation automatique (WOR-38), playbooks Ansible (WOR-44)
Authentification	Vitale	≤ 5 min	Sans objet : une session perdue oblige seulement à se reconnecter	Retour en même temps que l'API	Redémarrage ; après une rotation de secret, reconnexion des utilisateurs acceptée
WAF / point d'entrée	Vitale	≤ 5 min	Sans objet ; la configuration est versionnée	Pas de coupure pour un blocage d'IP	Liste d'IP bloquées rechargée à chaud (20 s maximum) ; les autres règles demandent un redémarrage
Journalisation et SIEM	Importante	≤ 1 h	**0** journal perdu	Aucune perte pendant les essais	Export des journaux (`scripts/export-logs.sh`) **avant** toute restauration
Consultation de l'historique	Importante	≤ 30 min	0	Retour en même temps que l'API	Même service que l'API
Supervision, détection ML, remédiation	Support critique	≤ 15 min	Sans objet ; le modèle est versionné	Hors périmètre de l'attaque	Relance par Docker Compose

Service	Criticité	RTO métier	RPO métier	Cible démonstrateur	Mécanisme prévu
Assistant RAG	Non critique	Meilleur effort	Sans objet ; l'index peut être reconstruit	—	Réindexation

4. Répartition du RTO sur la chaîne de reprise

Le RTO se décompose en étapes. Chaque pilier tient une partie du budget. Les durées sont des **hypothèses à confirmer** avec Reza (collecte), Léa (détection) et Michael (remédiation).

Étape	Marqueur de temps	Responsable	Budget attaque A	Budget attaque B
Début de l'impact → anomalie visible dans les métriques	T1 → T2	Infrastructure (collecte Prometheus toutes les 5 s)	≤ 5 s	≤ 5 s
Anomalie visible → alerte ML	T2 → T3	IA / Data (fenêtre de 60 s évaluée toutes les 10 s, alerte après trois fenêtres anormales consécutives)	≈ 30 s	≈ 30 s
Alerte → décision de remédiation	T3 → T4	Développement	≤ 5 s	≤ 5 s
Décision → action terminée	T4 → T5	Infrastructure / Développement	≤ 15 s (blocage d'IP au WAF ; redémarrage de l'API si besoin, jusqu'à 90 s)	≤ 2 min (blocage, recréation du conteneur)
Action terminée → transaction test réussie et données vérifiées	T5 → T6	Développement	≤ 10 s (jusqu'à 6 essais espacés de 3 s)	≤ 20 s
Total (RTO mesuré = T6 – T1)			**≤ 60 s**	**≤ 3 min**

T0 désigne le lancement de l'attaque sur le PC Attaquant. L'écart T0 → T1 dépend de l'attaque et ne compte pas dans le RTO.

> ***Point de vigilance pour le Clip Chrono :*** *60 s, c'est T0 → T6* ***et*** *l'affichage du résultat. La collecte passe déjà toutes les 5 s (`infra/prometheus/prometheus.yml`). La détection reste le poste le plus long : environ 30 s si trois évaluations consécutives suffisent, jusqu'à 80 s si la fenêtre de 60 s doit se remplir d'abord (`docs/runbooks/reprise.md`). Le passage de 2 à 3 fenêtres (21/10, pour réduire les fausses alertes) ajoute 10 s : la somme des budgets de l'attaque A atteint 65 s et dépasse les 60 s.* ***À mesurer en priorité avec Léa.***

5. La restauration après une injection (attaque B)

Ce que fait le code : le moteur recrée le conteneur de l'API, qui garde ses données en mémoire. Le service revient vite, mais **toutes les transactions sont perdues** (RPO non tenu). C'est le choix retenu pour l'instant ; les options ci-dessous restent celles à étudier pour tenir le RPO = 0.

Le scénario d'attaque (WOR-29) prévoyait un **retour à une copie instantanée (snapshot) propre de la base**. C'est incompatible avec un RPO de 0 : toutes les transactions légitimes validées entre le snapshot et la restauration seraient perdues.

Option	RPO obtenu	Effort	Avis
1. Snapshot périodique seul (par exemple toutes les 5 min)	Jusqu'à 5 min de transactions perdues	Faible	Simple, mais contredit l'objectif métier et provoque un incident majeur DORA (perte de données)
2. **Snapshot + rejeu des transactions légitimes** depuis un journal en ajout seul, en écartant celles qui viennent de la source bloquée	**0** si le rejeu est correct	Moyen	**Recommandé** : démontre l'objectif métier, et le tri entre légitime et malveillant est un bon argument pour le jury

Option	RPO obtenu	Effort	Avis
3. Restauration à un instant précis (avec PostgreSQL : archivage continu du journal des écritures), juste avant la première requête malveillante	Proche de 0, mais les transactions légitimes postérieures sont perdues	Moyen à élevé	Solution industrielle de référence, à citer dans les recommandations
4. Correction ciblée : annuler les seules lignes altérées, sans restauration globale	0	Élevé et risqué	Non retenu pour un prototype de 4 jours

Proposition : option 2 si l'équipe a le temps de brancher l'API sur PostgreSQL, option 3 citée comme cible industrielle. Dans tous les cas, **les journaux sont copiés avant la restauration** (`scripts/export-logs.sh`) : ce sont des preuves pour l'investigation et la notification.

6. Protocole de mesure (pour WOR-79)

Temps

Chaque marqueur est horodaté automatiquement, sur une **même horloge** (le PC Hôte ; le PC Attaquant est synchronisé dessus ou son décalage est noté avant chaque essai).

Marqueur	Source
T0 lancement de l'attaque	Journal du script d'attaque (WOR-58)
T1 première transaction en échec	Générateur de trafic normal : première réponse en erreur ou en dépassement de délai
T2 anomalie visible	Prometheus : premier point hors norme
T3 alerte ML	Journal du service d'inférence (WOR-53)
T4, T5 décision, action terminée	Événements publiés par le moteur de remédiation (WOR-38)
T6 reprise vérifiée	Moteur de remédiation : transaction test et contrôle de cohérence réussis

Données

1. Le générateur de trafic enregistre chaque transaction **confirmée** (identifiant, montant).
2. Après la reprise, on compare cette liste au contenu de la base.
3. **Transactions manquantes + transactions en double = RPO mesuré.** L'objectif est 0.

Grille de résultats

Trois essais minimum par attaque pendant les répétitions (WOR-78) ; détail des essais dans le journal du moteur.

Attaque	Détection T1 → T3	Reprise T3 → T6	RTO mesuré (pire des 3)	Transactions perdues ou dupliquées	Objectif tenu ?
A — déni de service	≈ 30 s (réglage : 3 fenêtres)	~7 s	**≈ 40 s**	0 (reprise par blocage, sans redémarrage)	Oui : ≤ 60 s (démo) et ≤ 5 min (métier)
B — injection	—	—	—	—	Non chronométrée : l'attaque vise une route inexistante

Résultats des répétitions du 22/10. La détection n'a pas été chronométrée séparément : la valeur est celle qu'impose le réglage du modèle.

On retient le **pire des trois essais**, pas la moyenne : un RTO se garantit, il ne s'espère pas.

7. Ce que le code permet de tenir au 22/10

- **RPO = 0 : pas tenable en l'état.** Dans le socle, l'API garde ses données en mémoire : un redémarrage (attaque A) ou une restauration (attaque B) efface toutes les transactions. Aucune sauvegarde.
- **Attaque A : reprise totale ≈ 40 s** aux répétitions du 22/10 (≈ 30 s de détection, ~7 s de reprise), sans transaction perdue ni intervention humaine (§6).

- **Vérification de la reprise (T6) : en place.** Le moteur passe une transaction de test complète à travers le WAF.
- **Durée affichée par le moteur :** elle part de la réception de l'alerte (T3 → T6). Le RTO se calcule depuis T1, avec le protocole du §6.

Détail et sources : `constat/etat-implementation.md`.

8. Ce que ces objectifs ne couvrent pas

- **Perte du PC Hôte** (panne, rançongiciel R7) : pas de deuxième site, donc pas de RTO tenable. La reprise passe par une réinstallation depuis le dépôt Git (WOR-49), sans objectif chiffré. C'est un risque résiduel assumé, à porter dans le PCA/PRA (WOR-71).
- **Sauvegardes sur le même hôte** : le RPO de 0 ne tient que si la base n'est pas détruite avec son hôte.
- **Faux positif du modèle ML** : une remédiation déclenchée à tort provoque elle-même une coupure, que le RTO doit absorber. C'est arrivé le 21/10 : deux fausses alertes d'injection ont déclenché deux restaurations (`ml/training/README.md`). À suivre pendant le chaos engineering (WOR-76).

PCA / PRA — Plateforme de paiement RESILIENCE-ALPHA

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans* `constat/etat-implementation.md`*.*

> *Ticket : WOR-71 · Responsable : Cédric · Version 0.3 du 21/10/2026*
>
> ***Statut :*** *les actions automatiques décrites sont celles du code (moteur de remédiation* `services/remediation/`*, playbooks* `ansible/playbooks/remediation/`*), relevées dans* `constat/etat-implementation.md` *v0.3. Chaque fiche sera confirmée après les essais (§9). Chaque fiche se lit seule : l'assistant RAG peut la citer isolément.*

1. Objet et périmètre

- **PCA (plan de continuité)** : comment le service de paiement **continue** pendant l'incident, en mode normal ou dégradé.
- **PRA (plan de reprise)** : comment on **revient** à un état sain et vérifié après l'incident, avec quelles données.

Dans le périmètre	Hors périmètre
Plateforme Docker Compose du PC Hôte : front, authentification, API de paiement, base de données, WAF, supervision, détection ML, moteur de remédiation, assistant RAG	Infrastructure du campus, postes des autres groupes
Incidents de disponibilité, d'intégrité, défaillances internes, perte de l'hôte	Données réelles : toutes les données sont fictives

2. Rôles et responsabilités

2.1 Dans l'entreprise fictive

Le **moteur de remédiation** intervient en premier, dans son périmètre d'actions. L'**opérateur d'astreinte** reprend la main s'il échoue. Le **responsable de la production** décide des restaurations et des modes dégradés ; le **RSSI** qualifie les incidents de sécurité et décide de la préservation des preuves ; le **directeur de crise** arbitre entre reprise rapide et investigation. La **conformité** classe l'incident selon DORA et notifie l'ACPR, la **communication** informe les partenaires, le **DPO** évalue l'atteinte aux données personnelles.

2.2 Dans l'équipe du démonstrateur

Membre	Ce qu'il porte dans ce plan
Michael (Développement)	Moteur de remédiation, vérification de la reprise, cohérence des données
Reza (Infrastructure)	Playbooks Ansible, sauvegardes et restauration, remise à l'état sain
Léa (IA / Data)	Détection ML, assistant RAG
Axel (Cybersécurité)	WAF, SIEM, secrets, préservation des journaux
Cédric (Gouvernance)	Ce plan, classification des incidents, notifications, retour d'expérience

3. Niveaux de réponse et escalade

Les seuils viennent du coût d'un arrêt (`services-critiques.md`).

Niveau	Déclencheur	Qui agit	Gravité
N0 — Automatique	Alerte ML ou SIEM	Moteur de remédiation seul ; l'opérateur est informé	G1
N1 — Astreinte	Échec de la remédiation, **ou** service non rétabli à 5 min, **ou** incident d'intégrité ou de confidentialité (même si la remédiation a réussi)	Opérateur d'astreinte, RSSI informé	G2

Niveau	Déclencheur	Qui agit	Gravité
N2 — Cellule de crise	Service non rétabli à 30 min, **ou** perte de données confirmée, **ou** compromission du moteur de remédiation	Directeur de crise, RSSI, conformité, communication	G3
N3 — Crise majeure	Service non rétabli à 2 h, **ou** perte du PC Hôte	Direction générale	G4

Règle de l'automate : l'automate fait **au plus 2 tentatives par incident** (action puis vérification). Après 2 échecs, il s'arrête, marque l'incident en échec et émet une escalade vers l'opérateur (N1, `docs/runbooks/reprise.md`). Une nouvelle alerte du même type pendant un incident en cours est fusionnée : seules les nouvelles sources sont bloquées. Il n'y a pas de limite d'un incident à l'autre. Après un redémarrage ou une restauration, le moteur **ignore toute nouvelle alerte pendant 180 s** (afflux de reconnexions des clients) ; il la journalise (`alert_suppressed`) sans agir.

4. Fiches scénarios

Fiche S1 — Déni de service sur l'API de paiement (attaque A)

- **Risque EBIOS :** R1 (hacktiviste, ER1) · gravité G4 si non traité
- **Signaux :** alerte ML de la famille `traffic` (débit, latence, erreurs, part de la première IP), après 3 fenêtres anormales consécutives
- **Actions automatiques :**
  1. **Blocage complet (403) au WAF** des sources qui pèsent au moins 20 % du trafic suspect ; la liste est rechargée à chaud
  2. Transaction de test ; si elle échoue, redémarrage de l'API
  3. Vérification : transaction de test réussie à travers le WAF
- **Données :** aucune restauration. Mais l'API du socle garde ses données en mémoire : le redémarrage (action 2) efface les transactions
- **Objectif :** RTO ≤ 5 min (cible démonstrateur ≤ 60 s) · RPO = 0 (pas tenable tant que l'API est en mémoire)
- **Escalade :** N1 après 2 tentatives en échec ; une nouvelle alerte pendant l'incident ne bloque que les nouvelles sources
- **Retour à la normale : pas de levée automatique** des blocages. L'opérateur lève la liste (`waf-unblock-all.yml`) ; la remise à l'état sain le fait aussi
- **Risques résiduels :**
  - L'attaquant change d'adresse : le blocage par source ne suffit plus (un vrai déni de service distribué exige une protection en amont, hors de portée du prototype)
  - Un partenaire légitime au fort volume peut être **bloqué** à tort, jusqu'à une action manuelle

Fiche S2 — Injection applicative et altération de données (attaque B)

- **Risque EBIOS :** R2 (cybercriminel, ER2) · gravité G4
- **Signaux :** alerte ML de la famille `injection` (motifs d'injection et blocages du WAF dans les journaux), après 3 fenêtres anormales consécutives. Une seule requête d'injection bloquée par le WAF suffit à la déclencher
- **Actions automatiques :**
  1. Blocage complet (403) de la source au WAF
  2. Constat d'intégrité : appel à `/health` (qui répond toujours « ok » : ce n'est pas un vrai contrôle)
  3. **Restauration à l'état sain connu** : recréation du conteneur de l'API. L'API étant en mémoire, elle repart des données de départ ; **toutes les transactions, légitimes comprises, sont perdues**
  4. Vérification : transaction de test réussie à travers le WAF
- **Pas automatisé :** copie des preuves (`scripts/export-logs.sh`, à lancer à la main **avant** la restauration si possible), rotation des secrets, validation humaine
- **Objectif :** RTO ≤ 5 min (cible ≤ 3 min) · RPO = 0 visé, **non tenu** par la restauration actuelle

- **Escalade :** automatique seulement en cas d'échec. Procédure : l'opérateur informe le RSSI de tout incident d'intégrité, même résolu ; la conformité évalue s'il est majeur au sens de DORA (une perte de données après un accès malveillant suffit)
- **Retour à la normale :** après correction de la vulnérabilité exploitée. D'ici là, la règle WAF reste active
- **Risques résiduels :**
  - La vulnérabilité est toujours dans le code : le WAF n'est qu'un pansement
  - Une porte dérobée a pu être déposée ; restaurer les données ne la supprime pas
  - Une alerte à tort efface quand même toutes les transactions (voir S4)
  - Une tentative d'injection que le WAF a bloquée déclenche quand même la restauration, donc la perte des transactions
  - Pendant les 180 s qui suivent la restauration, une nouvelle attaque n'est pas traitée automatiquement : l'opérateur surveille le tableau de bord
  - Sans export manuel, les preuves de l'attaque restent dans Loki, mais rien ne les protège d'une remise à l'état sain

Fiche S3 — Panne d'un service sans attaque

- **Couvre :** arrêt brutal d'un conteneur, saturation mémoire, base injoignable (testé par le chaos engineering, WOR-76), service de reconnaissance faciale indisponible (l'administrateur enrôlé ne peut plus se connecter au back-office)
- **Signaux :** contrôle de santé en échec, métriques absentes
- **Actions :** Docker Compose relance le conteneur (`healthcheck`) ; sinon, l'opérateur applique le runbook
- **Objectif :** RTO ≤ 5 min · RPO = 0
- **Escalade :** pas d'action automatique : le moteur ne réagit qu'aux alertes ML. L'opérateur relance le service (`docs/runbooks/reprise.md`)
- **Risques résiduels :** une cause profonde (disque plein, fuite mémoire) fera revenir la panne ; aucune connexion de secours au back-office n'est prévue si la reconnaissance faciale reste indisponible

Fiche S4 — Remédiation déclenchée à tort (faux positif du modèle ML)

- **Risque :** non intentionnel, cité dans `ebios-rm.md`
- **Signaux :** incident ouvert sans attaque en cours, sources bloquées qui sont des clients légitimes
- **Déjà survenu :** le 21/10 entre 12 h 36 et 13 h 34, sur du trafic légitime, 2 fausses alertes d'injection ont chacune déclenché une restauration (`ml/training/README.md`). Le détecteur a été réglé ensuite (3 fenêtres, seuil plus strict) : ce scénario reste possible, mais moins fréquent
- **Ce que fait le code :** une seule alerte ML suffit à déclencher toutes les actions, y compris la restauration. Il n'y a pas de second signal exigé (l'alerte combine déjà deux méthodes de détection, mais sur les mêmes données)
- **Conduite à tenir :** l'opérateur lève les blocages à tort (`waf-unblock-all.yml`) et note l'incident pour le réentraînement du modèle
- **Risques résiduels :** coupure provoquée par la défense elle-même ; après une fausse alerte d'injection, **perte de toutes les transactions**. La règle des deux signaux reste recommandée pour la production (`recommandations.md` REC-3)

Fiche S5 — Moteur de remédiation défaillant ou détourné

- **Risque EBIOS :** R3 (initié malveillant, ER6) · gravité G4
- **Signaux :** action hors du périmètre autorisé, appel au moteur depuis un composant non autorisé, rafale d'actions
- **Conduite à tenir :**
  1. Arrêter le moteur de remédiation
  2. Passer en **surveillance manuelle** : l'opérateur suit le tableau de bord et applique les fiches à la main
  3. Préserver les journaux du moteur

- **Escalade :** N2 immédiat
- **Prévention (en place) :** alertes signées HMAC, refus des alertes trop anciennes ou rejouées, liste fermée de playbooks, IP de l'hôte jamais bloquées, écoute locale, journal des événements. **Limite :** le moteur tourne sur l'hôte avec les droits Docker de l'utilisateur
- **Risques résiduels :** pendant l'arrêt du moteur, plus aucune reprise automatique ; on retombe sur un RTO manuel de 30 min ou plus

Fiche S6 — Suspicion de fuite de données

- **Risque EBIOS :** R5 (ER3) · gravité G3
- **Signaux :** volume de lecture anormal, requêtes inhabituelles sur la base, sortie de données vers une destination inconnue
- **Conduite à tenir :** couper l'accès de la source, préserver les preuves, **ne pas restaurer** (restaurer n'efface pas une fuite), prévenir le DPO
- **Escalade :** N2
- **Remarque :** non joué pendant la démo ; la réponse diffère d'une simple remise en service

Fiche S7 — Perte du PC Hôte (PRA)

- **Risque EBIOS :** R7 (rançongiciel) ou panne matérielle · gravité G4
- **Conduite à tenir :**
  1. Préparer une machine de remplacement (prérequis du README, WOR-49)
  2. Cloner le dépôt Git, redéployer avec Docker Compose et les playbooks Ansible
  3. Régénérer les secrets : Vault tourne sur le PC Hôte, son contenu est perdu avec lui. Le déploiement en crée de nouveaux quand `infra/.env` est absent
  4. Restaurer la dernière sauvegarde de la base, **si elle existe hors de l'hôte**
- **Objectif :** aucun objectif chiffré tenable avec un seul hôte
- **Risques résiduels : perte de toutes les données depuis la dernière copie hors hôte**. C'est le principal écart du démonstrateur (voir §8)

5. Mode dégradé

Quand la reprise automatique échoue, on maintient ce qui peut l'être :

Service	En mode dégradé	Interdit
Paiement	Limitation de débit stricte : on accepte moins de transactions plutôt qu'aucune	—
WAF	Mode de filtrage strict	**Le contourner** pour rétablir le trafic plus vite
Consultation	Lecture seule, voire suspendue pour soulager la base	—
Supervision	Surveillance manuelle sur le tableau de bord	—
Assistant RAG	Peut être arrêté pour libérer des ressources	Lui donner des droits d'action (WOR-57)

6. Choix de restauration

Élément	Où se trouve la référence saine	Comment on restaure	Fréquence	Responsable
Code et configuration des services	Dépôt Git, version taguée (WOR-80)	Redéploiement Docker Compose et Ansible	À chaque livraison	Reza

Élément	Où se trouve la référence saine	Comment on restaure	Fréquence	Responsable
Règles du WAF	Dépôt Git	Liste d'IP bloquées : rechargement à chaud ; autres règles : redémarrage du WAF	À chaque modification	Axel
Secrets	HashiCorp Vault, sur le PC Hôte	Rotation puis redémarrage des services concernés	Après tout incident d'intégrité	Axel
Données de l'API	**En mémoire dans le socle, aucune sauvegarde**	Recréation du conteneur : retour aux données de départ (`restore-service.yml`)	— (minimum souhaité : base PostgreSQL et `pg_dump` avant chaque essai)	Reza, Michael
Journaux	Loki, rétention 7 jours, sur le PC Hôte	**Jamais écrasés** ; copiés avant toute restauration avec `scripts/export-logs.sh` (à la main pour l'instant)	En continu	Axel
Modèle ML	Dépôt Git (`ml/`)	Rechargement de la version validée	À chaque entraînement	Léa
Index du RAG	Documents du dépôt	Réindexation	À chaque mise à jour des documents	Léa

> ***Ne pas confondre restauration et remise à l'état sain.*** *La remise à l'état sain (`scripts/reset-known-good.sh`, WOR-90) supprime les volumes pour rejouer un essai : elle efface les transactions,* ***mais aussi les journaux Loki et les métriques Prometheus****, donc les preuves. Elle ne doit* ***jamais*** *servir à reprendre après un incident réel.*

7. Communication et notification

Destinataire	Quand	Quoi	Qui
Opérateur d'astreinte	Dès N0	Alerte et état de la reprise sur le tableau de bord	Automate
Partenaires commerçants	Dès N1 si leurs transactions échouent	Nature de la gêne, heure de reprise estimée	Communication
ACPR, incident majeur au sens de DORA	Notification initiale dans les 4 h après la classification en majeur (et au plus tard 24 h après la détection), rapport intermédiaire sous 72 h après la notification initiale, rapport final sous 1 mois après le dernier rapport intermédiaire	Formulaire réglementaire, à partir des journaux du SIEM	Conformité
CNIL	Sous 72 h en cas de violation de données personnelles	Notification RGPD	DPO

Délais DORA : règlement délégué (UE) 2025/301. L'assistant RAG peut préparer un brouillon de notification à partir des journaux ; un humain le valide et l'envoie.

8. Risques résiduels après la reprise

Risque résiduel	Pourquoi il subsiste	Traitement proposé
L'attaquant revient, depuis une autre source	Le blocage est fait par source, sur un seul hôte	Protection anti-déni de service en amont (contexte industriel)
Vulnérabilité toujours présente après une injection	Le WAF bloque, mais le code n'est pas corrigé	Correctif et test de non-régression avant de lever le niveau N1
Porte dérobée ou altération ancienne	Restaurer les données ne nettoie pas tout	Analyse après incident à partir des journaux conservés
Hôte unique, sans sauvegarde hors site	Choix du prototype	Voir les limites industrielles, `recommandations.md` §3

Risque résiduel	Pourquoi il subsiste	Traitement proposé
Boucle de remédiation	Un incident persistant peut faire redémarrer un service en boucle	2 tentatives par incident, puis escalade ; pas encore de limite entre incidents
Faux positifs ML	Modèle entraîné sur du trafic normal seulement ; une alerte suffit à restaurer	Évaluer le modèle sur des attaques ; en production, deux signaux pour les actions destructrices (REC-3)
Droits élevés du moteur de remédiation	Nécessaires pour agir sur les conteneurs	Périmètre limité, authentification, journalisation (WOR-64)

9. Tests et maintien du plan

Test	Ticket	Scénarios vérifiés	Résultat
Premiers tests de restauration	WOR-75	S2, S7 (en partie)	Réalisés le 22/10 : fonctionnent comme décrit dans les fiches
Chaos engineering	WOR-76	S3, S4	Réalisé le 22/10 : fonctionne comme décrit dans les fiches
Démo complète répétée	WOR-78	S1	Attaque A reprise en ≈ 40 s, sans perte ni intervention humaine (`rto-rpo.md` §6). S2 non chronométré (attaque B sur une route inexistante)

Le plan est mis à jour après chaque test et après chaque incident (retour d'expérience, DORA art. 13).

10. Décisions en attente

1. Restauration après une injection : garder la recréation automatique (perte des transactions) ou ajouter une validation humaine et une vraie restauration de base : équipe, Cédric.
2. Brancher l'API du socle sur PostgreSQL et sauvegarder avant chaque essai : Reza, Michael.
3. Levée automatique des blocages, limite entre incidents, export des preuves avant restauration : Michael.
4. Ne pas restaurer quand le WAF a bloqué toutes les requêtes d'injection : Michael, Léa.
5. Connexion de secours au back-office si la reconnaissance faciale est indisponible : Axel, Michael.

Recommandations à la direction

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans `constat/etat-implementation.md`.*

> *Ticket : WOR-72 · Responsable : Cédric · Version 1.0 du 22/10/2026*
>
> *Synthèse des autres documents de `cible/`. Les preuves viennent des répétitions du 22/10 (`rto-rpo.md` §6) ; celles qui restent à produire sont marquées à mesurer.*

Synthèse pour la direction

1. **Un arrêt de paiement coûte peu pendant 5 minutes, puis le coût explose.** Une heure d'arrêt en pointe coûte environ 700 k€, essentiellement en pénalités contractuelles, et déclenche probablement une notification à l'ACPR au titre de DORA.
2. **La reprise automatique est donc l'investissement prioritaire.** Aux répétitions, le démonstrateur a rétabli le service en ≈ 40 s après un déni de service, vérifié par une transaction réelle et sans intervention humaine, contre 30 minutes ou plus pour une reprise manuelle.
3. **Elle ne suffit pas pour une attaque sur l'intégrité des données.** Une injection impose de préserver les preuves, de vérifier les données et de notifier, même si le service revient vite. En production, l'automate ne doit pas décider seul dans ce cas ; le démonstrateur le fait, pour la démonstration.
4. **Le démonstrateur repose sur un seul serveur.** C'est acceptable pour prouver le principe, pas pour la production : un deuxième site et des sauvegardes hors site sont la première dépense à prévoir.
5. **Nous demandons trois décisions** à la direction (dernière section).

0. Ce que les essais ont prouvé, et ce qui manque

Au 22/10, la chaîne détection → remédiation → vérification est codée et mesurée sur l'attaque A : reprise en ≈ 40 s, sans transaction perdue ni intervention humaine. Il manque la sauvegarde de la base, et dans le socle l'API garde ses données en mémoire : un redémarrage ou une restauration perd les transactions. Les écarts restants sont dans `constat/etat-implementation.md` §4.

1. Les arbitrages que nous avons faits

Arbitrage	Ce que nous avons choisi	Ce que nous avons écarté	Pourquoi	Ce que ça coûte
Quoi protéger d'abord	La chaîne de paiement : API, base, authentification, WAF	Protéger tous les services au même niveau	Ce sont eux qui portent le revenu et l'engagement envers les 200 partenaires	L'assistant RAG et la consultation passent après
Automatique ou humain	Pour la démo : tout automatique, un humain n'intervient qu'en cas d'échec (2 tentatives)	Faire valider les restaurations par une personne	Montrer une reprise sans intervention dans le temps imparti	Une alerte à tort peut déclencher une restauration ; en production, nous recommandons l'inverse (REC-2, REC-3)
Vitesse ou preuves	Restaurer au plus vite ; preuves exportées à la main (`scripts/export-logs.sh`)	Bloquer la reprise tant que la copie n'est pas faite	Garder le temps de reprise court	Si l'export est oublié, une remise à l'état sain efface les preuves
Scénario principal de la démo	Déni de service (risque R1) ; injection en second scénario	Une seule attaque	R1 est le risque le plus élevé ; le second scénario montre que la réponse dépend du type d'incident	Deux chaînes de remédiation à construire et tester

Arbitrage	Ce que nous avons choisi	Ce que nous avons écarté	Pourquoi	Ce que ça coûte
Hébergement	Tout sur un hôte, sans dépendance à un cloud	Services managés	Imposé par le cadre du projet ; évite aussi le risque lié à un prestataire	Point de défaillance unique, assumé et documenté
Restauration après injection	Retour à l'état de départ : recréation du conteneur de l'API	Snapshot et rejeu des transactions légitimes	Simple et rapide à démontrer	**Toutes les transactions sont perdues** : RPO = 0 non tenu (REC-6). Et une tentative d'injection bloquée par le WAF déclenche quand même la restauration
Référentiels	DORA, EBIOS RM ; NIS2 en grille de lecture	PCI DSS	Aucune donnée de carte manipulée	—

2. Recommandations

Chaque recommandation repose sur un élément **vérifiable**, issu de nos essais ou de nos documents.

Priorité 1 — à décider maintenant

#	Recommandation	Élément vérifiable	Effort
REC-1	**Généraliser la reprise automatique** aux incidents de disponibilité sur toute la chaîne de paiement	Reprise totale mesurée : ≈ 40 s sur l'attaque A ; interventions humaines pendant les répétitions : 0 (WOR-78)	Moyen
REC-2	**Garder un humain dans la boucle pour les incidents d'intégrité** : blocage automatique, mais restauration et notification validées par une personne. Le démonstrateur restaure automatiquement : c'est un choix de démo, pas une cible de production	Fiche S2 du PCA ; délai entre l'alerte et la décision humaine, à mesurer en exercice	Faible
REC-3	**Exiger deux signaux concordants** avant toute action destructrice (le démonstrateur agit sur une seule alerte ML)	Faux positifs du modèle : environ une fausse alerte de trafic toutes les 7 h ; le 21/10, deux fausses alertes d'injection ont chacune déclenché une restauration à tort (`ml/training/README.md`)	Faible
REC-4	**Encadrer le moteur de remédiation** comme un bien critique : périmètre d'actions limité, appels authentifiés, journalisation	Revue des droits du moteur (WOR-64) ; essai d'appel depuis un composant non autorisé, qui doit être refusé	Moyen

Priorité 2 — dans les 6 mois

#	Recommandation	Élément vérifiable	Effort
REC-5	**Sauvegardes hors site et immuables**, avec restauration testée chaque trimestre	Temps de restauration mesuré ; nombre de transactions perdues à la restauration	Moyen
REC-6	**Restauration de la base à un instant précis** (archivage continu des journaux de la base), pour tenir zéro transaction perdue après une injection	Essai d'injection suivi d'une restauration, sans transaction légitime perdue	Moyen
REC-7	**Procédure de notification DORA** : classification, brouillon préparé à partir des journaux, validation, envoi à l'ACPR	Exercice de notification chronométré face aux délais réglementaires	Faible
REC-8	**Programme annuel de tests de résilience** couvrant tous les risques retenus dans l'EBIOS, pas seulement le scénario de démo	Taux de couverture des risques R1 à R8 par au moins un test	Moyen

Priorité 3 — dans les 12 mois

#	Recommandation	Élément vérifiable	Effort
REC-9	**Deuxième site** (ou deuxième zone de disponibilité), avec bascule testée	Durée de bascule mesurée	Élevé
REC-10	**Protection anti-déni de service en amont** du WAF, pour les attaques distribuées	Essai de charge distribuée	Moyen, souvent sous forme de service
REC-11	**Gestion des prestataires TIC selon DORA** : registre, clauses contractuelles, plans de sortie	Registre à jour, contrats revus	Moyen

Ordre de grandeur pour arbitrer : un seul incident de 30 minutes à 2 heures en pointe coûte environ 700 k€ (`services-critiques.md`). Les priorités 1 et 2 visent à ce qu'il ne se produise pas ; la priorité 3 protège contre la perte complète du site.

3. Ce que nous ferions autrement en contexte industriel

Dans le démonstrateur	En production
Un seul hôte, sauvegardes locales	Deux sites, sauvegardes hors site et immuables
Un modèle ML entraîné sur 7,3 h de trafic de la plateforme	Plusieurs semaines de données réelles, réentraînement régulier, suivi de la dérive du modèle
Deux scénarios d'attaque, joués par l'équipe	Programme de tests annuel, tests d'intrusion par des testeurs indépendants (TLPT si l'entité est désignée)
Notification décrite sur papier	Procédure exercée, astreinte conformité, modèles de rapports ACPR
Aucun contrat avec les prestataires TIC	Registre d'information, clauses DORA, plans de sortie (REC-11)
Données fictives, sauf les visages des membres de l'équipe enrôlés pour la double authentification (consentement écrit, effacement après la démo ; cadre : `docs/governance/rgpd-biometrie.md`)	Analyse d'impact sur la protection des données (AIPD) avant de traiter des données réelles, obligatoire pour la biométrie (RGPD art. 9 et 35) ; double authentification proposée aussi aux clients ; détection du vivant pour la reconnaissance faciale
Rôles fictifs (directeur de crise, RSSI, DPO)	Rôles nommés, suppléants, exercices de crise deux fois par an
Hypothèses financières fictives	Analyse d'impact sur l'activité menée avec les métiers et la finance
Gouvernance portée par l'équipe projet	Cadre de risque TIC approuvé par l'organe de direction, comme l'exige DORA (art. 5)

4. Limites de cette analyse

- Les chiffres financiers sont **fictifs**. Ils donnent des ordres de grandeur, pas un budget.
- Les seuils DORA et les délais de notification sont repris de notre lecture des textes, à confirmer par un juriste.
- Les temps de reprise ne vaudront preuve qu'une fois mesurés (WOR-79). Si les mesures ne tiennent pas les objectifs, les recommandations REC-1 et REC-3 seront revues.

5. Décisions demandées à la direction

1. **Valider le principe d'une reprise automatique** pour les incidents de disponibilité, avec validation humaine pour les incidents d'intégrité (REC-1, REC-2).
2. **Accepter le risque résiduel de l'hôte unique** pour la phase pilote, et **inscrire au budget** les sauvegardes hors site (REC-5) puis le deuxième site (REC-9).

3. **Désigner les responsables** des rôles du PCA (directeur de crise, conformité, astreinte) et planifier un premier exercice de crise.

État d'implémentation face à la gouvernance

> ***Statut : constat.*** *Ce document décrit ce qui a été vérifié dans le code, avec ses sources. Les objectifs et les plans sont dans `cible/`.*

> *Responsable : Cédric · Version 0.5 du 22/10/2026 · Relevé sur `main` après la fusion des PR #14 à #21, #24, #26 (moteur de remédiation, détection ML, tableau de bord), #29 (Pay&Go déplacé dans `services/payment-api/`), #47 à #58, #61 (consigne additionnelle 1, tableaux Grafana, détecteur réentraîné, note RGPD sur la biométrie)*
>
> *Les documents de gouvernance ont été écrits avant le code. Ce registre dit, pour chaque hypothèse importante, si le code la **confirme**, la **contredit** ou ne l'a **pas encore implémentée**. Il est à mettre à jour après chaque fusion importante et avant les essais (WOR-73).*

1. Ce qui existe au 22/10/2026

Composant	État	Où
API de paiement (Node.js) : sessions, idempotence des paiements, métriques Prometheus, journaux JSON, limitation à 60 requêtes par minute et par client, 12 essais de connexion par 15 min, **verrouillage de l'identifiant après 2 échecs** (5 min, puis 10, 15…)	Implémentée. **Dans le socle, elle garde tout en mémoire** (voir §2)	`services/payment-api/backend/`, `infra/docker-compose.yml`
Front web et page d'administration	Implémentés	`services/payment-api/front/`
Base PostgreSQL de l'application	Implémentée, **sans sauvegarde**, utilisée seulement par le déploiement autonome de Pay&Go	`services/payment-api/infra/docker-compose.yml`
Socle : Vault, PostgreSQL, Prometheus (collecte toutes les 5 s, moteur de remédiation compris), Loki (rétention 7 jours), Grafana	Implémenté ; l'API y est collectée	`infra/docker-compose.yml`
WAF ModSecurity + OWASP CRS, en mode bloquant, seul point d'entrée publié	Implémenté, l'API est derrière lui. Injections bloquées en 403. Liste d'IP bloquées rechargée à chaud (`nginx -s reload`, 20 s maximum) ; les autres règles demandent un redémarrage	`infra/waf/`
Détection ML : Isolation Forest par famille (`traffic` → déni de service, `injection` → injection), fenêtre de 60 s au pas de 10 s, alerte après **3 fenêtres anormales**	Implémentée, en conteneur. Réentraînée le 21/10 sur 7,3 h de trafic légitime de la plateforme : environ une fausse alerte de trafic toutes les 7 h, aucune d'injection en validation. **Taux et délai de détection des attaques A et B pas encore mesurés**	`ml/inference/`, `ml/training/README.md`
Moteur de remédiation : reçoit les alertes signées, exécute une liste fermée de playbooks Ansible, vérifie par une transaction de test. **Ignore les nouvelles alertes pendant 180 s** après un redémarrage ou une restauration	Implémenté, **sur l'hôte** (pas en conteneur), avec les droits Docker de l'utilisateur	`services/remediation/`, `scripts/start-remediation.sh`
Playbooks de remédiation : blocage d'IP au WAF, redémarrage, restauration (recréation du conteneur), levée des blocages	Implémentés	`ansible/playbooks/remediation/`

Composant	État	Où
Tableau de bord d'incident (WOR-40), servi par le moteur	Implémenté	`services/dashboard/`
Tableaux de bord Grafana : SOC / SIEM, plateforme et paiements, détection et reprise (chronologie, durée de reprise, escalades)	Implémentés ; aucune règle d'alerte Grafana	`infra/grafana/provisioning/`
Remise à l'état sain	Implémentée ; **efface base, journaux, métriques et Vault**, lève les blocages	`scripts/reset-known-good.sh`
Export des journaux de Loki vers un dossier de preuves (empreintes SHA-256)	Implémenté, **à lancer à la main** : ni la remise à l'état sain ni le moteur ne l'appellent	`scripts/export-logs.sh`
Durcissement des conteneurs (non-root, `cap_drop`, limites de ressources)	Implémenté	`infra/docker-compose.yml`
Double authentification du back-office (consigne additionnelle 1) : identifiant, puis visage par la caméra, puis mot de passe ; enrôlement par l'administrateur	Implémentée (WOR-95 à 97). Exigée seulement pour les comptes dont le visage est enrôlé ; 2 échecs de visage verrouillent l'identifiant. Empreintes faciales conservées dans le volume `face-data`, jamais les images	`services/face-recognition/`, `services/payment-api/`
Objet connecté : LED de l'ESP8266 qui reflète l'état de la connexion au back-office (consigne additionnelle 1)	Implémenté (WOR-99, 100). Lit `/v1/device/led`, **ouvert si `DEVICE_TOKEN` n'est pas défini**, ce qui est le cas dans le socle	`iot/led-indicator/`
Assistant RAG en lecture seule	Implémenté	`rag/docker-compose.rag.yml`
Scripts d'attaque A et B	Implémentés ; **l'attaque B vise encore une route inexistante** (voir §3)	`attack/`

La chaîne attaque → détection → remédiation → vérification existe maintenant de bout en bout dans le socle. Aux répétitions du 22/10, l'attaque A a été reprise en ≈ 40 s (≈ 30 s de détection, ~7 s de l'alerte à la transaction de test), sans transaction perdue ni intervention humaine.

2. Hypothèses de la gouvernance confrontées au code

Point principal : dans le socle, l'API garde ses données en mémoire. Elle n'utilise PostgreSQL que si `STORAGE_DRIVER=postgres` (`services/payment-api/backend/api/src/main.ts`), variable absente pour `payment-api` dans `infra/docker-compose.yml`. Le playbook de restauration le dit lui-même : « payment-api est en mémoire : le recréer le remet dans l'état du seed » (`restore-service.yml:2`). Conséquences :

- un redémarrage (attaque A) ou une restauration (attaque B) efface **toutes** les transactions, légitimes comprises ;
- le RPO = 0 est impossible, même sans attaque ;
- la « restauration » n'est pas une restauration de données, c'est un retour à l'état de départ.

Hypothèse	Document	Verdict	Constat
Aucune transaction confirmée perdue (RPO = 0)	`cible/rto-rpo.md`	**Contredit**	API en mémoire : chaque redémarrage ou restauration perd les transactions. Aucune sauvegarde

Hypothèse	Document	Verdict	Constat
Après une injection : snapshot puis rejeu des transactions légitimes	`cible/rto-rpo.md` §5, `cible/pca-pra.md` S2	**Contredit**	Le moteur recrée le conteneur (`restore-service.yml`) : pas de snapshot, pas de rejeu
Un humain valide la restauration après une injection	`cible/recommandations.md` REC-2, `cible/pca-pra.md` S2	**Contredit**	Restauration automatique sur une seule alerte (`engine.py`)
Deux signaux concordants avant une action destructrice	`cible/pca-pra.md` S4	**Non implémenté**	Une seule alerte ML suffit. Elle combine Isolation Forest et enveloppe en « ou », ce qui n'est pas deux signaux indépendants
Au plus 3 remédiations en 10 minutes par service	`cible/pca-pra.md` §3	**Contredit**	2 tentatives par incident puis escalade (`config.py:44`, `MAX_ACTION_ROUNDS`), sans limite d'un incident à l'autre
Déni de service : limitation de débit par source, levée après 30 min	`cible/pca-pra.md` S1	**Contredit**	**Blocage complet en 403** de toute IP pesant au moins 20 % du trafic suspect (`ml/inference/service.py:36`). Pas de levée automatique : seule la remise à l'état sain débloque
Reprise vérifiée par une transaction réelle	`cible/rto-rpo.md` §1	**Confirmé**	Connexion, intention de paiement, code, paiement réussi, à travers le WAF (`services/remediation/remediation/probe.py`). Jusqu'à 6 essais espacés de 3 s
Constat d'intégrité avant restauration	`cible/pca-pra.md` S2	**Partiel**	Un simple appel à `/health`, qui répond toujours « ok »
Journaux copiés avant toute restauration	`cible/pca-pra.md` S2, §6	**Partiel**	`scripts/export-logs.sh` existe mais n'est appelé ni par le moteur ni par la remise à l'état sain
Moteur de remédiation protégé (droits limités, ordres authentifiés)	`cible/pca-pra.md` S5, `cible/recommandations.md` REC-4	**En grande partie confirmé**	Alertes signées HMAC avec contrôle de fraîcheur et anti-rejeu, liste fermée de playbooks, IP de l'hôte jamais bloquées, écoute locale. Mais il tourne sur l'hôte avec les droits Docker
Escalade vers un humain	`cible/pca-pra.md` §3	**Partiel**	Seulement après échec (événement `escalation`). Pas d'escalade systématique après une injection
Collecte toutes les 5 s, détection en 30 à 80 s	`cible/rto-rpo.md` §4	**Confirmé pour l'attaque A**	≈ 30 s aux répétitions du 22/10, valeur imposée par le réglage (3 fenêtres au pas de 10 s). Attaque B non chronométrée
Verrouillage progressif des comptes	`cible/ebios-rm.md` R4, `cible/recommandations.md`	**Confirmé**	2 échecs puis 5 min, durée croissante, y compris avec le bon mot de passe pendant le verrou (`InMemoryLoginAttemptTracker.ts`). Verrous en mémoire : un redémarrage les efface

Hypothèse	Document	Verdict	Constat
Double authentification	`cible/ebios-rm.md` R4	**Confirmé pour le back-office**	Visage exigé avant le mot de passe pour les comptes enrôlés. Les comptes clients n'en ont pas
Durée de reprise affichée = RTO	`cible/rto-rpo.md` §1	**À corriger à la mesure**	Le moteur compte depuis la réception de l'alerte (T3 → T6), pas depuis le début de l'impact (T1)
Assistant RAG sans accès privilégié	`cible/pca-pra.md` §5	**Confirmé**	Conteneur en lecture seule, sans droits ni accès au moteur Docker
Hôte unique, aucune sauvegarde hors de l'hôte	`cible/nis2-dora.md`, `cible/pca-pra.md` S7	**Confirmé**	Inventaire Ansible local uniquement
Secrets dans HashiCorp Vault	`cible/ebios-rm.md` BS9	**Partiellement contredit**	Vrai pour le socle et la clé HMAC (fichier hors Git). Le déploiement autonome de Pay&Go garde son mot de passe de base en clair et sa clé OpenAI en variable d'environnement

3. Nouveaux constats à intégrer à l'analyse de risques (WOR-67)

Constat	Risque concerné	Gravité du constat
Restauration automatique qui efface les transactions légitimes, déclenchée par une seule alerte ML	ER6 (remédiation en erreur), ER2 altération	**Élevée** : un faux positif d'injection suffit à perdre toutes les transactions
Blocage complet d'une IP qui pèse 20 % du trafic, sans levée automatique	ER6, R1	Moyenne : un partenaire légitime au fort volume peut être coupé jusqu'à une action manuelle
Moteur de remédiation sur l'hôte avec les droits Docker de l'utilisateur	R3, ER6	Moyenne : le bien support le plus sensible n'est pas isolé ; non mentionné dans `docs/governance/durcissement.md`
Port de la base de l'application (55432) publié sur toutes les interfaces, mot de passe en clair dans le dépôt	R5 exfiltration	**Élevée** : la base est joignable depuis le réseau `RESILIENCE-G6`
L'assistant de discussion de l'application envoie les messages à OpenAI	Fuite de données (ER3), prestataire TIC tiers (DORA art. 28)	Moyenne : données fictives, mais incohérent avec un assistant « local »
Code de vérification des paiements fixe (`123456`) **et renvoyé dans la réponse de l'API** (`demoCode`)	R4 usurpation, R2 fraude	Moyenne en démo, critique en production
Vault avec une seule part de clé, clé de descellement et jeton root dans un fichier de l'hôte	R3, R7	Moyenne : acceptable pour un prototype, à citer en limite
Attaque B sur `/api/pay`, qui n'existe pas (les routes sont en `/v1/…`)	Validité des essais	**Élevée** : l'attaque B ne touche pas l'API ; elle ne prouve que le blocage par le WAF
Pas de solde ni de débit dans le modèle de données (intentions de paiement uniquement)	ER2 altération	Faible : le risque devient « paiements altérés ou dupliqués »

Constat	Risque concerné	Gravité du constat
Risque déjà survenu : le 21/10 entre 12 h 36 et 13 h 34, sur du trafic légitime, 2 fausses alertes d'injection ont chacune déclenché une restauration de payment-api (`ml/training/README.md`). Corrigé en réglant le détecteur	ER6, ER2	**Élevée** : confirme le constat de la première ligne. Le réglage réduit la fréquence, pas la conséquence
Une seule requête d'injection, **pourtant bloquée par le WAF**, suffit à déclencher l'alerte, donc la restauration qui efface les transactions	ER6, R2	**Élevée** : n'importe qui peut provoquer la perte des transactions avec une requête
Pendant 180 s après un redémarrage ou une restauration, le moteur ignore toute alerte	R1, R2	Moyenne : une attaque relancée dans cette fenêtre n'est pas traitée (elle est journalisée `alert_suppressed`)
Verrouillage par identifiant : quelques essais au mauvais mot de passe bloquent le compte légitime, de plus en plus longtemps	R1 (disponibilité du back-office)	Faible : contournement de l'usurpation au prix d'un déni de service ciblé, classique et accepté
Le back-office répond `FACE_VERIFICATION_REQUIRED` avant de vérifier le mot de passe	R4	Faible : révèle qu'un compte existe et a un visage enrôlé
Empreintes faciales des membres de l'équipe : données biométriques **réelles** (RGPD art. 9), alors que le reste des données est fictif	ER3, RGPD	Moyenne : consentement écrit des personnes enrôlées et effacement après la démo (bouton « révoquer » du back-office, `POST /v1/admin/faces/revoke`, ou purge du volume `face-data`). Cadre du traitement : `docs/governance/rgpd-biometrie.md`
La reconnaissance faciale ne vérifie pas que le visage est vivant : une photo de l'administrateur peut passer (`services/face-recognition/`)	R4	Moyenne : le mot de passe reste exigé. À citer en limite ; en production, détection du vivant
Si le service de reconnaissance faciale tombe, l'administrateur ne peut plus se connecter (erreur 502)	R1	Faible : dépendance à documenter dans le PCA (connexion de secours)

4. Ce qu'il faut avant les essais

Par ordre de priorité, pour que la démo prouve ce que la gouvernance affirme :

1. **Brancher l'API du socle sur PostgreSQL** (`STORAGE_DRIVER=postgres`) et faire un `pg_dump` avant chaque essai. Sans cela, chaque remédiation perd les transactions. Infrastructure, Développement.
2. **Évaluer le modèle sur des fenêtres d'attaque** : taux et délai de détection, faux positifs. IA / Data.
3. **Recâbler l'attaque B** sur une route en `/v1/…`. Cybersécurité.
4. **Arbitrer la restauration après injection** : automatique (code actuel) ou validée par un humain (REC-2). Au minimum, ne pas restaurer quand le WAF a bloqué toutes les requêtes d'injection. Puis aligner le code ou la gouvernance. Gouvernance, Développement.
5. **Appeler `scripts/export-logs.sh`** avant toute restauration et avant la remise à l'état sain. Développement, Infrastructure.
6. **Chronométrer T1 → T6** sur trois essais par attaque (`cible/rto-rpo.md` §6). Tous.
7. **Fermer le port 55432 et passer les secrets de l'application dans Vault.** Le sujet exige aussi qu'aucun secret ne figure en clair dans l'archive du code (§7.4) : le mot de passe `payandgo` de `services/payment-api/infra/docker-compose.yml` doit en sortir avant le rendu. Cybersécurité.
8. **Recueillir le consentement écrit** des personnes dont le visage est enrôlé et prévoir l'effacement après la démo (`docs/governance/rgpd-biometrie.md`). Gouvernance, Cybersécurité.
9. **Corriger `docs/governance/rgpd-biometrie.md`**, qui affirme « aucun secret en clair », tant que le mot de passe `payandgo` reste dans `services/payment-api/infra/docker-compose.yml`. Cybersécurité.